

SSL Playbook — b2bwebapp.com

Unified Knowledge Base & Runbook for Certificate Installation, Debugging, and Automation.

Overview

This document is a complete single-source SSL playbook containing:

- End-to-end steps from CSR creation to installation in Kubernetes.
- Debugging steps for all common SSL/Ingress issues.
- Lessons learned from real production incident.
- cert-manager automation setup.
- Automated CI-based secure rotation pipeline.
- One-page on-call checklist.

Prerequisites

- Access to DNS provider (GoDaddy).
- Access to Azure Kubernetes Cluster (AKS).
- kubectl configured with prod namespace.
- CA-issued certificate files (leaf + intermediate).
- Private key stored securely.

Files received from CA

Usually:

- leaf.crt
- intermediate.crt
- fullchain.crt

Full chain must ALWAYS be: leaf first → intermediates next.

Creating Key + CSR

```
openssl genpkey -algorithm RSA -out b2b.key -pkeyopt rsa_keygen_bits:2048
openssl req -new -key b2b.key -out b2b.csr -config csr.conf
CSR SAN config: [req]
req_extensions = v3_req distinguished_name = req_distinguished_name
[req_distinguished_name] [v3_req] subjectAltName = @alt_names [alt_names]
DNS.1 = b2bwebapp.com DNS.2 = *.b2bwebapp.com DNS.3 = app.b2bwebapp.com DNS.4 =
api.b2bwebapp.com
```

Assemble Fullchain

```
cat leaf.crt intermediate.crt > b2b.fullchain.crt
grep -c 'BEGIN CERTIFICATE' b2b.fullchain.crt
```

Create TLS Secret in prod

```
kubectl -n prod create secret tls b2b-tls --cert=b2b.fullchain.crt
--key=b2b.key --dry-run=client -o yaml | kubectl apply -f -
```

Ingress TLS Reference

```
spec:
  tls:
  - hosts:
    - app.b2bwebapp.com
    - api.b2bwebapp.com
    secretName: b2b-tls
```

Restart Ingress Controller

```
kubectl -n ingress-nginx rollout restart deployment ingress-nginx-controller
```

Verify Secret

```
kubectl -n prod get secret b2b-tls -o jsonpath='{.data.tls\.crt}' | base64
--decode | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
```

Check Logs

```
POD=$(kubectl -n ingress-nginx get pods -l app.kubernetes.io/component=controller
-o jsonpath='{.items[0].metadata.name}') kubectl -n ingress-nginx logs $POD -c
```

```
controller --tail=300 | egrep -i 'Adding secret|no valid PEM|fake certificate'
```

Test SNI

```
openssl s_client -connect :443 -servername app.b2bwebapp.com | openssl x509  
-noout -subject -issuer
```

Curl Test

```
curl -v --resolve app.b2bwebapp.com:443: https://app.b2bwebapp.com/
```

Common Issues & Fixes

1. Fake Certificate: Cause: secret mismatch or bad PEM. Fix: recreate secret & restart. 2. no valid PEM formatted block: Cause: corrupted cert or wrong order. Fix: rebuild fullchain. 3. LB serving old cert: Cause: controller not reloaded. Fix: restart ingress controller. 4. Azure Front Door in path: Must upload cert there too.

Manual Renewal Steps

```
kubectl -n prod create secret tls b2b-tls --cert=new_fullchain.crt  
--key=new.key --dry-run=client -o yaml | kubectl apply -f - kubectl -n  
ingress-nginx rollout restart deployment ingress-nginx-controller
```

cert-manager Automation

```
apiVersion: cert-manager.io/v1 kind: Certificate metadata: name: b2b-cert  
namespace: prod spec: secretName: b2b-tls dnsNames: - app.b2bwebapp.com  
- api.b2bwebapp.com issuerRef: name: letsencrypt-staging kind:  
ClusterIssuer
```

CI/CD Rotation Pipeline

Steps: 1. Download cert from CA or Vault. 2. Create TLS secret: `kubectl -n prod create secret tls b2b-tls --cert=cert.pem --key=key.pem --dry-run=client -o yaml | kubectl apply -f -` 3. Restart controller. 4. Smoke test using openssl and curl.

On-call Checklist

1. `dig app.b2bwebapp.com`. 2. `kubectl -n prod get secret b2b-tls`. 3. Decode cert → verify expiry & SANs. 4. Check ingress secretName. 5. Check logs for PEM errors. 6. openssl SNI test. 7. curl test. 8. Fix → recreate secret & restart. 9. Record new expiry.

End of SSL Playbook — b2bwebapp.com